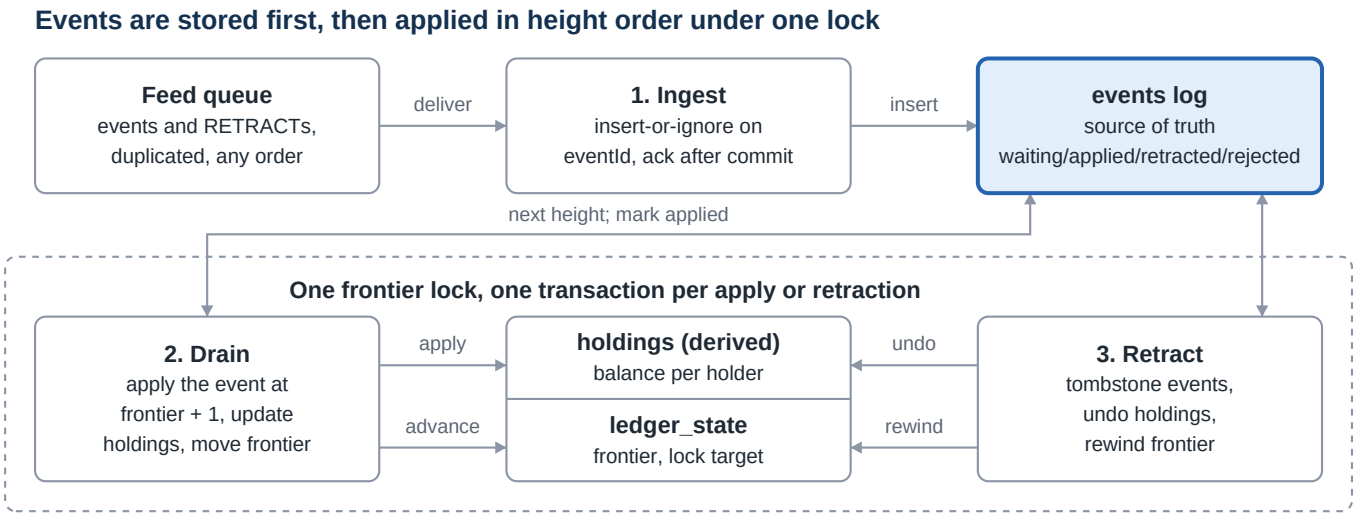
Ownership Ledger Service: Architecture Note

Senior Full Stack Engineer second-round exercise | Seema | 2026-11-11

The service keeps an append-only event log as the source of truth and applies events strictly in height order under one lock. Finalized and provisional holdings are computed at read time, never stored.

Flow



Reads: holdings plus applied events above finalized_through, one transaction. Assumes one event per height.

Data model, and why

Table	Holds
`events`	Append-only log and source of truth, never deleted from: unique `eventId`, `kind`, `height`, asset, from, to, units, `fromHeight` (retractions), `status` (`waiting`, `applied`, `retracted`, `rejected`)
`holdings`	Derived balance per asset and holder with a rule that `units >= 0`, changed in the same transaction as every apply and undo
`ledger_state`	One row and the lock target: contiguous `frontier`, `highestSeen` and `finalized_through`
`assets`	Issued supply, changed by an issuance or by retracting one

Finalized and provisional amounts are never stored, so no job can leave them stale. An applied event is finalized when its height is at or below `finalized_through`, and provisional above that. The log gives tombstones and audit. `holdings` gives a one-lookup negative check and can be rebuilt from the log.

Finality never moves backwards. A retraction rewinds the frontier, so `frontier - 12` computed live could fall: a finalized figure would drop, and a later retraction could reach heights already reported final. So the line is stored as a high-water mark, `finalized_through = GREATEST(finalized_through, frontier - 12)`, raised under the frontier lock on every frontier write. The retraction guard and the reads both use it. Only the line is stored.

Ordering, duplicates and retraction

One scenario: the frontier is at 4470, INV-012 sends 30 units of AST-7 to INV-044, and INV-044 passes them on.

Colors: waiting | applied | retracted | ignored | alert

#	What arrives	What the service does	Frontier
1	`evt_B` at 4472, before 4471	Out of order: stored `waiting`, acknowledged	4470
2	`evt_A` at 4471	Stored, then the drain applies 4471 and 4472	4472
3	`evt_A` again, second instance, same instant	Unique `eventId`: one insert wins, the copy is ignored	4472
4	`RETRACT` from 4472	Same lock: `evt_B` tombstoned, effect undone, frontier rewinds	4471
5	`evt_B` redelivered	Hits its tombstone and is ignored	4471
6	`evt_C` at 4472, corrected, new id	Stored and applied	4472
7	`evt_E` at 4473 overspends its sender	Drain stops and alerts, never skips	4472

- **Height decides order**, never arrival. Events wait in the log until the drain reaches them.
- **The database decides duplicates.** Insert-or-ignore on `eventId`, acknowledged after commit. Each apply is its own transaction, so a crash loses at most the event in flight, which stays `waiting`.
- **Retraction rewinds, never deletes.** Tombstones stop resurrection. A retraction reaching final heights (at or below `finalized_through`) is rejected and alerted.

Edge rules: the same id with different contents replaces a tombstone but alerts on a live row. A retraction briefly blocks ingest, so each event is swept by it or arrives after it. Any instance drains after each ingest and on a timer, so a crash cannot strand an event.

Most likely production failure, and detection

A stalled frontier (a missing height, or an event that would push a holder negative) raises no error. Reads stay stale and finality stops, so a distribution run quietly uses old numbers.

- **Lag alert:** `highestSeen - frontier` warns at 4 heights and is critical at 10, below the 12-height finality horizon.
- **Stall alert:** the frontier has not moved for 5 minutes while events are waiting.
- **Waiting events:** their count and the age of the oldest are exposed on `/metrics`.
- **Reconciliation:** `reconcile()` rebuilds `holdings` from the log and checks supply per asset. The test runs it; in production, run it hourly and after every retraction. Responses carry `appliedThrough`, so readers see staleness too.

Thresholds are configurable defaults.

If the feed were 1000x larger

The first limit is the serial drain: one lock and one event per transaction cap throughput at one lock's commit rate.

#	What breaks	Change	Trade-off
1	The serial drain	Batch the drain, then partition by asset, each with its own frontier and lock	The feed must say a height is complete per asset, and a retraction becomes a barrier across all partitions
2	Ingest volume	Add queue partitions and ingest handlers	None for ordering, since ingest only inserts rows
3	Read cost and log size	Snapshot of finalized balances, log partitioned by height, read replicas	A second write path, and replica lag shown by `appliedThrough`
4	Reconciliation time	Check one height range at a time	A full rebuild stays as an occasional deep check

Apply them in this order, each only once the previous limit is measured, not guessed.

What I deliberately did not build

- **A real queue:** the test simulates at-least-once, unordered, concurrent delivery through one handler, so correctness does not depend on queue features.
- **Automatic gap repair:** the brief gives no replay interface, so a gap alerts a human, and a skip is a human-approved, audited action.
- **Scheduled reconciliation, snapshot fold, per-asset frontiers:** the check exists as a function the test calls. The other two are scale features whose risk is not justified at this size.
- **Auth, a UI, a metrics backend, epoch handling:** out of scope, or not provided by the feed.

Assumptions

- **One event per height.** Two events at one height stop the drain and alert.
- **No empty heights:** every height carries an event, so a missing height is a gap.
- **A corrected event arrives after its RETRACT,** with a new id or a retracted one (a retracted id with new contents replaces the tombstone). The brief gives no epoch or sequence number.
- **Finality is measured from the contiguous frontier,** not the highest height seen: final once height <= `frontier - 12`, and it stays final after a rewind.
- **No replay interface:** a gap alerts a human.

The concurrency test

The most dangerous scenario is a retraction racing with out-of-order, duplicated delivery. It is the only one where both invariants can pass while the ledger is wrong: a stale transfer conserves units and rarely drives anyone negative. So the test replays a feed with a retraction and a corrected sequence, shuffled and duplicated, through concurrent workers on one database, and compares the result with a single-threaded oracle. It asserts no negatives, supply equal to issuance, holdings equal to the oracle, and one row per `eventId`. Switching off each of the six protections in turn (unique id, frontier lock, tombstones, ingest barrier, negative check, finality guard) must turn it red.

Part A: Gap at 4472

We hold 4471 and 4473, but not 4472.

Choice	What it does	Verdict
A. Apply 4473 now, fix later	Shows 4473 immediately	Rejected: can break the no-negative rule and needs undo logic
B. Drop 4473	Keeps the ledger clean	Rejected: loses a valid event the queue will not resend
C. Hold 4473 as `waiting`	Applies only through 4471	**Chosen:** nothing wrong is ever shown

Under choice C:

1. 4473 arrives: store it as `waiting` and acknowledge it. The frontier stays at 4471.
2. 4472 arrives: the drain applies 4472, then 4473, one transaction each.
3. The gap stays open: alert at 4 heights past it, critical at 10, below the 12-height finality horizon.
4. A height is never skipped automatically. A skip is a human-approved, audited action.

Endpoints meanwhile: holdings through 4471 only, with `appliedThrough: 4471`, `gapAt: 4472` and `waitingCount: 1`. The 4473 event is in no balance and is not called provisional.

Why C: 4473 may spend units that 4472 delivers, and a stale labelled answer is recoverable where a wrong balance that triggered a payout is not. The cost is that a permanent gap blocks everything behind it, hence the alert.

Part B: Flagging unusual movements

Rules flag, AI explains, a human decides.

```
Ledger → Rules → Flag → Evidence pack (RAG) → Reviewer → Decision
```

Build first	Fixed rules that raise flags, such as "INV-044 went from 0 to 18% of AST-7's supply in 2 heights". Then an evidence pack that saves the reviewer 20 to 30 minutes of lookups per flag.
Never let AI decide	Raising, clearing or escalating a flag, or any action on an account. The model only writes the summary. The reviewer's decision and written reason become a past case.
How I'd know it works	Rules are replayed on past incidents (catches vs false flags). The right past case appears in the top 3. Every summary passes the citation check. Time per flag drops.

What the reviewer sees:

> INV-044 received 4,500 units from INV-012 and INV-031 at heights 4471–4472 [evt_8831, evt_8832]. Policy 4.2 requires enhanced review above 15% [policy 4.2]. Similar case: C-118, cleared as a family-trust transfer [case C-118].

Three guards:

1. Numbers come from SQL, never from retrieval, because a compliance figure must be exact.
2. RAG searches only policy documents and past cases. Past cases are matched on rule type and the numbers first and on text last, because embeddings can't tell 18% from 80%.
3. Code checks every id and number in the summary against the evidence, and drops the summary if any one doesn't match.